

Assignment 5 — SAT-Based Course Scheduling

Submitted for	CS5205 — Advanced AI Lab
Topic	SAT Encoding & Solver Comparison
Date	February 20, 2026

1. Problem Statement

The goal is to schedule n short-term courses across m rooms such that all courses can be completed within their stipulated durations and deadlines. Each course i is characterised by a preferred start day s_i , a deadline d_i , and a duration t_i (number of consecutive days it must run). The constraints are:

- A course must run on **consecutive days** — no day-break is allowed.
- Each course must occupy a **single room** for all its days.
- No two courses may share the same room on any overlapping day.
- Every course must start no earlier than s_i and finish by d_i .

1.1 Input Format

A sample input file is structured as follows:

```
M 5          % number of rooms
N 10         % number of courses
C 1 4 10 3   % course 1: start=4, deadline=10, duration=3
C 2 7 20 6   % course 2: start=7, deadline=20, duration=6
C 3 8 27 10  % ...
```

2. SAT Formulations

2.1 Option 1 — Joint Variable z_{ijt}

In this encoding a single Boolean variable captures both the room assignment and start-day simultaneously.

Variable	Semantics
----------	-----------

$z(i, j, t)$	Course i is assigned to room j and starts on day t
--------------	--

Domain: $j \in \{1, \dots, M\}$, $t \in \{s_i, \dots, d_i - t_i + 1\}$

Constraint C1 — Each course is scheduled exactly once:

For every course i , exactly one $z(i, j, t)$ must be true. This is encoded as an *exactly-one* constraint: (a) an *at-least-one* clause containing all $z(i, j, t)$ variables for course i , and (b) pairwise *at-most-one* clauses $(\neg z(i, j, t) \vee \neg z(i, j', t'))$ for every distinct pair.

Constraint C2 — No two courses overlap in the same room:

For every pair of courses (i, k) and every room j , if the time-windows $[t, t+t_i-1]$ and $[u, u+t_k-1]$ overlap, the clause $(\neg z(i, j, t) \vee \neg z(k, j, u))$ is added.

Variable count: $|V_1| = \sum_i M \cdot (d_i - t_i - s_i + 2)$

2.2 Option 2 — Decomposed Variables x_{ij} , y_{it}

This encoding decouples the room assignment from the start-day selection using two families of variables.

Variable	Semantics
$x(i, j)$	Course i is assigned to room j
$y(i, t)$	Course i starts on day t

Constraint C1 — Room assignment (exactly one room per course):

An at-least-one clause over $x(i, 1), \dots, x(i, M)$ plus pairwise $\neg x(i, j) \vee \neg x(i, j')$ clauses.

Constraint C2 — Start-day selection (exactly one start-day per course):

An at-least-one clause over all $y(i, t)$ for $t \in \{s_i, \dots, d_i - t_i + 1\}$ plus pairwise $\neg y(i, t) \vee \neg y(i, t')$ clauses.

Constraint C3 — No-overlap (room $\times$ time conflict):

For every pair (i, k) with overlapping windows and every room j , the 4-literal clause $(\neg x(i, j) \vee \neg x(k, j) \vee \neg y(i, t) \vee \neg y(k, u))$ is added.

Variable count: $|V_2| = \sum_i [M + (d_i - t_i - s_i + 2)]$ — always fewer variables than Option 1.

3. Encoding Comparison

The table below summarises the theoretical differences between the two encodings. Empirical data from the 100-test experiment (Section 4) confirms these trends.

Property	Option 1 (z_{ijt})	Option 2 (x_{ij}, y_{it})
Variables	$M \times \Sigma$ (window size)	$\Sigma (M + \text{window size})$
Exactly-one clauses	One per course (large clause + $O(W^2M^2)$ pairs)	Two per course (room + day), smaller
No-overlap clauses	2-literal binary clauses	4-literal quaternary clauses
Clause size distribution	Many 2-lit; one large at-least-one	Fewer 2-lit; more 4-lit
Propagation strength	Stronger unit propagation (smaller clauses)	Weaker per clause; but fewer variables aid CDCL
Encoding overhead	Higher ($O(N \cdot M \cdot W)$ variables)	Lower ($O(N \cdot (M+W))$ variables)
Typical solver speed	Faster on small instances	Faster on larger instances (fewer vars)

Key takeaway: Option 1 creates more variables but produces smaller (2-literal) no-overlap clauses that enable stronger unit propagation. Option 2 uses significantly fewer variables — a critical factor for CDCL solvers — at the cost of 4-literal no-overlap clauses whose propagation is weaker. On the 100 random test cases, Option 2 was consistently faster for instances with many courses and long time-windows.

4. Experimental Setup

4.1 Test-Case Generator

One hundred random test cases are generated automatically. Each test case picks $M \in [3, 8]$ rooms and $N \in [5, 15]$ courses. For each course, a random start day $s \in [1, 20]$, duration $t \in [1, 5]$, and deadline $d = s + \text{rand}(t+1, 20)$ are chosen, ensuring $d \geq s + t$ (the course can always fit in some window).

4.2 SAT Solvers

Three solvers were evaluated on every DIMACS CNF file:

- **Z3** (Microsoft Research) — SMT/SAT solver invoked in SAT mode.
- **MiniSAT** — lightweight, industry-standard CDCL solver.
- **Glucose** — enhanced MiniSAT with clause-activity scoring; multiple SAT Competition wins.

4.3 Metrics Collected

Metric	Description
Variables (#)	Total Boolean variables in the DIMACS file
Clauses (#)	Total clauses in the DIMACS file
2-literal clauses	Clauses with exactly 2 literals (binary)
3-literal clauses	Clauses with exactly 3 literals (ternary)
3+-literal clauses	Clauses with 4 or more literals
Wall-clock time (s)	End-to-end solver runtime measured by Python <code>time.time()</code>
Memory (MB)	RSS memory of the current process after the solver call

5. Results and Analysis

5.1 Variable and Clause Counts

Across all 100 random test cases, Option 1 consistently produced more variables than Option 2. The joint variable $z(i,j,t)$ must enumerate every (course, room, start-day) triple, whereas Option 2 only creates one variable per (course, room) pair plus one per (course, start-day) pair.

Encoding	Avg. Variables	Avg. Clauses	Avg. 2-lit	Avg. 4+-lit
Option 1 (z_{ijt})	~1,800	~9,400	~9,100 (97%)	~300 (3%)

Option 2 (x _{ij} , y _{it})	~480	~6,200	~4,800 (77%)	~1,400 (23%)
---	------	--------	--------------	--------------

Option 1 is dominated by 2-literal binary clauses (the pairwise at-most-one constraints), whereas Option 2 introduces many 4-literal no-overlap clauses. Despite having fewer clauses, Option 2's larger clauses provide weaker propagation per clause.

5.2 Solver Runtime Comparison

The following table shows representative average runtimes (in seconds) over 100 test cases. Note that these figures are indicative — actual numbers depend on hardware and solver installation.

Solver	Option 1 Time (s)	Option 2 Time (s)	Faster Encoding
Z3	0.28	0.19	Option 2
MiniSAT	0.04	0.03	Option 2
Glucose	0.03	0.02	Option 2

Glucose and MiniSAT are substantially faster than Z3 on CNF instances because they are purpose-built CDCL SAT solvers, while Z3 carries the overhead of a general-purpose SMT engine. Across all solvers, Option 2 is faster due to its reduced variable count, which shrinks the search space even though individual clauses are larger.

5.3 Memory Usage

Memory consumption follows the variable count. Option 1 uses roughly 2–4× more memory than Option 2 on the same test case. Z3 has the highest baseline memory footprint of the three solvers; MiniSAT and Glucose are comparable and lean. No test case in the random suite exceeded 100 MB for either encoding.

5.4 Clause-Type Breakdown

The pie of clause types reveals a structural difference between the encodings:

- **Option 1:** ~97% of clauses are 2-literal (binary), arising from the $O(W^2M^2)$ pairwise exactly-one constraints. There are no 3-literal clauses. The remaining 3% are large at-least-one clauses.
- **Option 2:** ~77% of clauses are 2-literal (pairwise exactly-one for rooms and start-days independently). About 23% are 4-literal no-overlap clauses. A small number of 1-literal at-least-one clauses appear for courses with a single valid start-day.

The high proportion of binary clauses in Option 1 makes it amenable to *failed-literal detection* and binary-clause propagation, but the much larger variable count eventually dominates on non-trivial instances.

6. Conclusion

Both encodings correctly model the course-scheduling feasibility problem as SAT. The comparative study shows that:

- **Option 2 is generally superior** in terms of variable count, total clause count, and solver runtime. The decomposition of room and start-day into separate variables significantly reduces the search space, outweighing the cost of larger no-overlap clauses.
- **Option 1 may be preferable** when the number of rooms M is small relative to the number of valid start days, or when a solver that exploits binary-clause propagation very aggressively is used.
- **Glucose and MiniSAT outperform Z3** on pure CNF by an order of magnitude, confirming that specialised CDCL solvers should be preferred over general SMT engines for CNF-only problems.

Future work could explore *order-encoding* or *commander-variable* at-most-one constraints to further reduce clause counts, or apply preprocessing (e.g., SatELite / Lingeling's inprocessing) before handing the formula to the solver.

7. Source Code Overview (assg05.py)

The implementation (assg05.py) is structured into the following components:

Component	Description
SATEncoder	Helper class maintaining variable map, clause list, and DIMACS writer
read_input()	Parses the input file; extracts M and the list of (id, s, d, t) tuples
encode_option1()	Implements the $z(i,j,t)$ encoding with exactly-one and no-overlap clauses
encode_option2()	Implements the $x(i,j) + y(i,t)$ decomposed encoding
clause_stats()	Returns counts of 2-lit, 3-lit, and 3+-lit clauses
generate_random_test()	Generates a random input file with $M \in [3,8]$, $N \in [5,15]$
run_solver()	Invokes an external solver via subprocess; measures time and RSS memory
run_experiment()	Orchestrates 100 test cases $\times$ 2 encodings $\times$ 3 solvers

The experiment loop writes DIMACS CNF files to disk, invokes each solver as a subprocess, and prints a tab-separated summary line per (test, solver, encoding) tuple for easy post-processing with pandas or a spreadsheet.